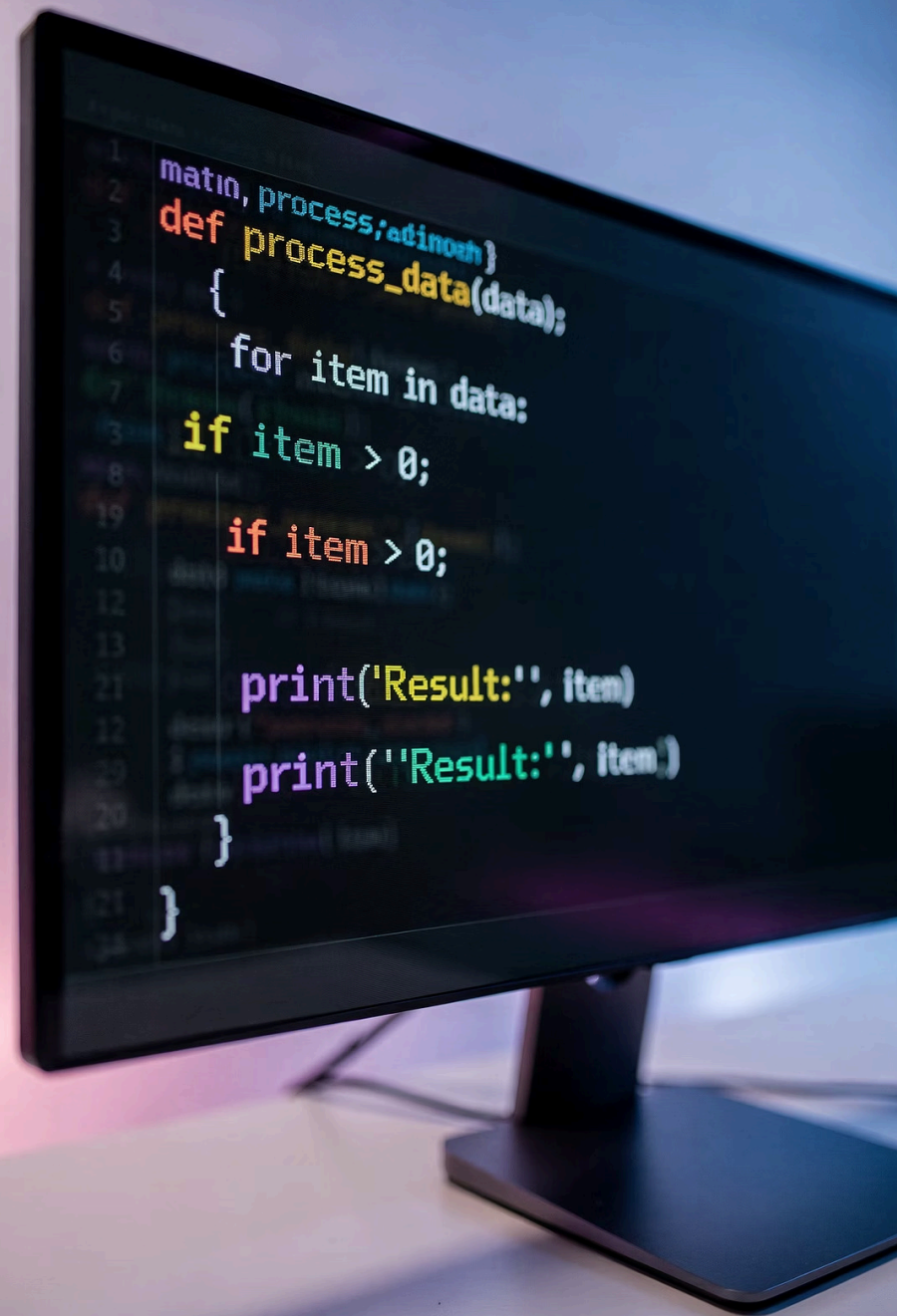


Estruturas de Repetição

Bem-vindo à aula de **Algoritmos**! Hoje vamos explorar dois dos recursos mais poderosos da programação: os laços de repetição **for** e **while** em Python. Eles permitem que você execute blocos de código múltiplas vezes sem precisar reescrevê-los, tornando seus programas mais eficientes, elegantes e poderosos.

ALGORITMOS II

PYTHON



Objetivo das Estruturas de Repetição

A ideia central por trás dos laços de repetição é simples, mas transformadora: **repetir instruções automaticamente** sem que o programador precise escrever o mesmo código diversas vezes. Imagine que você precisa imprimir os números de 1 a 100 – sem laços, seriam 100 linhas de código. Com um laço, são apenas 2 ou 3 linhas.

Eficiência

Reduza dezenas ou centenas de linhas repetidas a um bloco compacto e reutilizável de código.

Automatização

Execute tarefas repetitivas – leituras, cálculos, validações – de forma automática e controlada.

Escalabilidade

O mesmo código que processa 5 itens pode processar 5 milhões, sem nenhuma alteração adicional.

Em Python, as duas estruturas de repetição principais são o `while` e o `for`. Cada uma tem sua forma ideal de uso, e ao longo desta aula você aprenderá a escolher a certa para cada situação.

O Laço `while`

O `while` é um laço condicional: ele **executa um bloco de código enquanto uma condição for verdadeira**. A cada iteração, a condição é verificada novamente. Se ainda for verdadeira, o bloco executa de novo. Quando a condição se torna falsa, o laço termina e o programa continua normalmente.

Como funciona o `while`?

O Python avalia a condição antes de cada execução do bloco. Se a condição for `True`, o bloco executa. Se for `False`, o laço encerra imediatamente – sem executar o bloco nem mais uma vez.

Quando usar o `while`?

Use o `while` quando **não se sabe de antemão quantas vezes** o laço vai repetir. Por exemplo: ficar pedindo uma senha até o usuário digitar a correta, ou processar dados até o fim de um arquivo.

📌 ⚠️ Atenção: se a condição nunca se tornar falsa, o laço nunca termina – isso é chamado de **loop infinito**, um dos erros mais comuns para iniciantes!

Exemplo Prático: `while` em Ação

Código

```
i = 0
while i < 5:
    print(i)
    i += 1
```

O programa começa com `i = 0`. Enquanto `i` for menor que 5, ele imprime o valor de `i` e depois soma 1 a ele. Quando `i` chega a 5, a condição `i < 5` vira falsa e o laço para.

Saída no terminal

```
0
1
2
3
4
```

Passo a passo

01

`i = 0` → condição `0 < 5` é **True** → imprime 0 → `i` vira 1

02

`i = 1` → condição `1 < 5` é **True** → imprime 1 → `i` vira 2

03

`i = 4` → condição `4 < 5` é **True** → imprime 4 → `i` vira 5

04

`i = 5` → condição `5 < 5` é **False** → **laço encerrado**

O Laço `for`

O `for` é um laço de iteração: ele **percorre cada elemento de uma sequência** – como uma lista, uma string, ou um intervalo de números – e executa o bloco de código uma vez para cada elemento. Diferente do `while`, o `for` já sabe quantas vezes vai repetir antes mesmo de começar.



Iteração Automática

O `for` avança automaticamente para o próximo elemento a cada iteração. Você não precisa criar uma variável contadora nem incrementá-la manualmente – o Python cuida disso para você.



Funciona com qualquer sequência

Listas, tuplas, strings, dicionários, arquivos e qualquer objeto iterável podem ser percorridos com `for`. É a forma mais idiomática e pythônica de repetição quando o conjunto de dados é conhecido.



Quando preferir o `for`?

Sempre que você souber de antemão **quais elementos serão processados** ou **quantas repetições ocorrerão**. Para iterar sobre listas de dados, faixas numéricas ou coleções, o `for` é a escolha certa.

Exemplo Prático: `for` em Ação

Código

```
for i in range(5):  
    print(i)
```

O `for` percorre cada valor gerado por `range(5)`, que produz os inteiros de 0 a 4. A cada passo, o valor atual é armazenado em `i` e impresso.

Saída no terminal

```
0  
1  
2  
3  
4
```

Comparando with while

Perceba que o resultado é idêntico ao exemplo com `while`! A diferença está na sintaxe e na intenção. O `for` é mais conciso e elimina a necessidade de declarar e incrementar a variável contadora manualmente, reduzindo chances de erro.



O `for` com `range()` é a forma mais comum de repetir um bloco um número fixo de vezes em Python.

A Função `range()`

A função `range()` é a principal parceira do laço `for` em Python. Ela gera uma sequência de números inteiros de forma eficiente, sem precisar criar uma lista na memória. Sua sintaxe completa é:

```
range(inicio, fim, passo)
```

1

inicio

O valor onde a sequência começa (inclusive).
Padrão: 0 se omitido. Exemplo: `range(2, 10)` começa em 2.

2

fim

O valor onde a sequência termina (**exclusive** – o fim nunca é incluído). Exemplo: `range(0, 5)` gera 0, 1, 2, 3, 4.

3

passo

O incremento entre cada número. **Padrão: 1**. Pode ser negativo para contar de trás para frente. Exemplo: `range(0, 10, 2)` gera 0, 2, 4, 6, 8.

Exemplos práticos

```
range(5)      # 0, 1, 2, 3, 4  
range(1, 6)   # 1, 2, 3, 4, 5  
range(0, 10, 2) # 0, 2, 4, 6, 8  
range(10, 0, -1) # 10, 9, 8, ..., 1
```

Dica importante

O `range()` não cria uma lista – ele gera os valores sob demanda. Isso o torna extremamente eficiente mesmo para intervalos enormes, como `range(1_000_000)`, sem consumir muita memória.

Boas Práticas com Laços de Repetição

Escrever laços funcionais é apenas o começo. Para criar código limpo, seguro e fácil de manter, é fundamental seguir boas práticas. Os dois pilares mais importantes para iniciantes são: **evitar loops infinitos** e **controlar corretamente a variável de iteração**.

Evitar Loop Infinito

Um loop infinito ocorre quando a condição do `while` nunca se torna falsa. O programa trava e nunca termina. Sempre garanta que algo dentro do laço modifica a condição de saída.

```
#
```


Aplicações Práticas dos Laços

Os laços de repetição são a base de praticamente todo algoritmo útil. Veja três das aplicações mais comuns que você encontrará como programador:



Contagem

Use laços para contar elementos que satisfazem uma condição. Por exemplo, contar quantos números pares existem em uma lista, ou quantas vezes uma letra aparece em uma palavra.

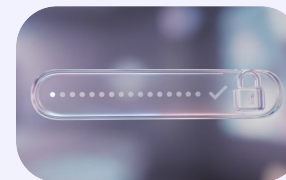
```
contagem = 0
for n in range(1, 11):
    if n % 2 == 0:
        contagem += 1
print(contagem) # 5
```



Soma e Acumulação

Acumule valores ao longo das iterações usando uma variável acumuladora. É uma das técnicas algorítmicas mais fundamentais – usada em médias, totais e estatísticas.

```
total = 0
for i in range(1, 6):
    total += i
print(total) # 15
```



Validação de Entrada

Use `while` para repetir a solicitação de dados até que o usuário forneça uma entrada válida – senha correta, número positivo, opção de menu válida, entre outros.

```
senha = ""
while senha != "1234":
    senha = input("Senha: ")
print("Acesso liberado!")
```

Executando o Programa

Como rodar no terminal

```
python aula_repeticao.py
```

Salve seu código em um arquivo com extensão `.py` e execute o comando acima no terminal (Prompt de Comando no Windows, Terminal no Linux/macOS). O Python irá interpretar e executar cada linha do seu arquivo.

Resumo do que aprendemos

- **while:** repete enquanto a condição for verdadeira – ideal quando não sabemos quantas iterações ocorrerão.
- **for:** percorre sequências – ideal para iterar sobre coleções ou intervalos numéricos com `range()`.
- **range():** gera sequências numéricas com início, fim (exclusivo) e passo configuráveis.
- **Boas práticas:** sempre controle sua variável de iteração e evite loops infinitos.
- **Aplicações:** contagem, soma acumulada e validação de entradas são os usos mais comuns.



🚀 Próximo passo: pratique criando seus próprios laços! Tente escrever um programa que some todos os números de 1 a 100 usando `for` e depois com `while`.